

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет физико-математических и естественных наук

Кафедра прикладной информатики и теории вероятностей

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 9.

Дисциплина: Архитектура компьютера

Студент: Павлушина Виктория Александровна

Группа: НКАбд-05-25

МОСКВА 2025 г.

Оглавление

1.Цель работы	3
2.Выполнение лабораторной работы	4
2.1. Реализация подпрограмм в NASM.....	4
2.2. Отладка программ с помощью GDB	6
2.3. Добавление точек останова.....	9
2.4. Работа с данными программы в GDB.....	11
2.5. Обработка аргументов командной строки в GDB.....	14
3.Задание для самостоятельной работы	16
4.Вывод.....	19

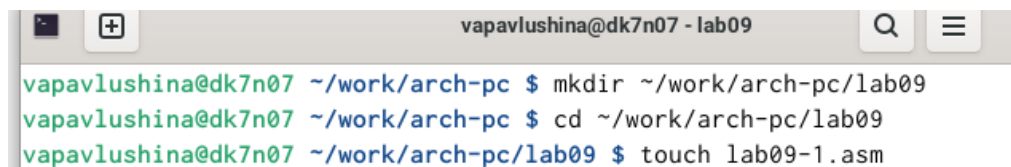
1.Цель работы

Приобретение навыков написания программ с использованием подпрограмм. Знакомство с методами отладки при помощи GDB и его основными возможностями.

2.Выполнение лабораторной работы

2.1. Реализация подпрограмм в NASM

Создадим каталог для выполнения лабораторной работы № 9, перейдём в него и создадим файл *lab09-1.asm*:



```
vapavlushina@dk7n07 - lab09
vapavlushina@dk7n07 ~/work/arch-pc $ mkdir ~/work/arch-pc/lab09
vapavlushina@dk7n07 ~/work/arch-pc $ cd ~/work/arch-pc/lab09
vapavlushina@dk7n07 ~/work/arch-pc/lab09 $ touch lab09-1.asm
```

Рисунок 1 Создание папки и файла для ЛР №9

В качестве примера рассмотрим программу вычисления арифметического выражения $f(x) = 2x + 7$ с помощью подпрограммы *_calcul*.

В данном примере *x* вводится с клавиатуры, а само выражение вычисляется в подпрограмме. Создадим файл *lab09-1.asm* с текстом листинга 9.1, оттранслируем и запустим:

```
vapavlushina@dk7n07 ~/work/arch-pc/lab09 $ nasm -f elf lab09-1.asm
vapavlushina@dk7n07 ~/work/arch-pc/lab09 $ ld -m elf_i386 -o lab09-1 lab09-1.o
vapavlushina@dk7n07 ~/work/arch-pc/lab09 $ ./lab09-1
Введите x: 3
2x+7=13
```

Рисунок 2 Запустили программу файла *lab09-1.asm*

```
%include 'in_out.asm'
SECTION .data
msg: DB 'Введите x: ',0
result: DB '2x+7=',0
SECTION .bss
x: RESB 80
res: RESB 80
SECTION .text
GLOBAL _start
_start:
;-----
; Основная программа
;-----
mov eax, msg
call sprint
mov ecx, x
mov edx, 80
call sread
mov eax, x
call atoi
call _calcul ; Вызов подпрограммы _calcul
mov eax, result
call sprint
mov eax, [res]
call iprintf
call quit
;-----
; Подпрограмма вычисления
; выражения "2x+7"
_calcul:
mov ebx, 2
mul ebx
add eax, 7
mov [res], eax
ret ; выход из подпрограммы
```

Рисунок 3 Листинг 9.1

Изменим текст программы, добавив подпрограмму `_subcalcul` в подпрограмму `_calcul`, для вычисления выражения $f(g(x))$, где x вводится с клавиатуры, $f(x) = 2x + 7$, $g(x) = 3x - 1$. Т.е. x передается в подпрограмму `_calcul` из нее в подпрограмму `_subcalcul`. Для этого создадим копию файла с именем `lab09-01.asm`:

```
vapavlushina@dk7n07 ~/work/arch-pc/lab09 $ touch lab09-01.asm
vapavlushina@dk7n07 ~/work/arch-pc/lab09 $ nasm -f elf lab09-01.asm
vapavlushina@dk7n07 ~/work/arch-pc/lab09 $ ld -m elf_i386 -o lab09-01 lab09-01.o
vapavlushina@dk7n07 ~/work/arch-pc/lab09 $ ./lab09-01
Введите x: 3
f(g(x)) = 2(3x-1)+7 = 23
vapavlushina@dk7n07 ~/work/arch-pc/lab09 $
```

Рисунок 4 Изменили программу, оттранслировали и запустили

```
%include 'in_out.asm'
SECTION .data
msg: DB 'Введите x: ',0
result: DB 'f(g(x)) = 2(3x-1)+7 = ',0
SECTION .bss
x: RESB 80
res: RESB 80
SECTION .text
GLOBAL _start
_start:
;-----
; Основная программа
;-----
mov eax, msg
call sprint
mov ecx, x
mov edx, 80
call sread
mov eax, x
call atoi
call _calcul ; Вызов подпрограммы _calcul
mov eax, result
call sprint
mov eax, [res]
call iprintLF
call quit

;-----
; Подпрограмма вычисления f(g(x))
; где f(x) = 2x + 7, g(x) = 3x - 1
; Вход: eax = x
; Выход: [res] = f(g(x)) = 2(3x-1)+7
_calcul:
push eax ; Сохраняем x в стеке
call _subcalcul ; Вызываем подпрограмму для вычисления g(x)
; Теперь в eax = g(x) = 3x-1
mov ebx, 2
mul ebx ; eax = 2 * g(x)
add eax, 7 ; eax = 2*g(x) + 7
mov [res], eax ; Сохраняем результат
pop eax ; Восстанавливаем регистр
ret

;-----
; Подпрограмма вычисления g(x) = 3x - 1
; Вход: eax = x (через стек из _calcul)
; Выход: eax = g(x) = 3x - 1
_subcalcul:
mov ebx, 3
mul ebx ; eax = 3 * x
sub eax, 1 ; eax = 3x - 1
ret
```

Рисунок 5 Текст программы lab09-01

2.2. Отладка программ с помощью GDB

Создадим файл *lab09-2.asm* с текстом программы из Листинга 9.2. (Программа печати сообщения Hello world!):

```
SECTION .data
msg1: db "Hello, ",0x0
msg1Len: equ $ - msg1
msg2: db "world!",0xa
msg2Len: equ $ - msg2
SECTION .text
global _start
_start:
mov eax, 4
mov ebx, 1
mov ecx, msg1
mov edx, msg1Len
int 0x80
mov eax, 4
mov ebx, 1
mov ecx, msg2
mov edx, msg2Len
int 0x80
mov eax, 1
mov ebx, 0
int 0x80
```

Рисунок 6 Текст листинга 9.2.

```
vapavlushina@dk7n07 ~/work/arch-pc/lab09 $ touch lab09-2.asm
```

Рисунок 7 Создали файл

Получим исполняемый файл. Для работы с GDB в исполняемый файл необходимо добавить отладочную информацию, для этого трансляцию программ необходимо проводить с ключом ‘-g’

```
vapavlushina@dk7n07 ~/work/arch-pc/lab09 $ nasm -f elf -g -l lab09-2.lst lab09-2.asm
vapavlushina@dk7n07 ~/work/arch-pc/lab09 $ ld -m elf_i386 -o lab09-2 lab09-2.o
```

Рисунок 8 Транслируем файл

Загрузим исполняемый файл в отладчик gdb:

```
vapavlushina@dk7n07 ~/work/arch-pc/lab09 $ gdb lab09-2
Exception caught while booting Guile.
Error in function "load-thunk-from-memory":
missing DT_GUILLE_ENTRY
gdb: warning: Could not complete Guile gdb module initialization from:
/usr/share/gdb/guile/gdb/boot.scm.
Limited Guile support is available.
Suggest passing --data-directory=/path/to/gdb/data-directory.
GNU gdb (Gentoo 16.3 vanilla) 16.3
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-pc-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://bugs.gentoo.org/>.
Find the GDB manual and other documentation resources online at:
  <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab09-2...
(gdb) run
Starting program: /afs/.dk.sci.pfu.edu.ru/home/v/a/vapavlushina/work/arch-pc/lab09/lab09-2
Hello, world!
```

Рисунок 9 Загружаем файл в отладчик

Для более подробного анализа программы установим брейкпоинт на метку `_start`, с которой начинается выполнение любой ассемблерной программы, и запустим её.

```
(gdb) break _start
Breakpoint 1 at 0x8049000: file lab09-2.asm, line 9.
```

Рисунок 10 Поставили точку остановки

Посмотрим дисассимилированный код программы с помощью команды **disassemble** начиная с метки `_start`:

```

(gdb) disassemble _start
Dump of assembler code for function _start:
=> 0x08049000 <+0>:      mov     $0x4,%eax
      0x08049005 <+5>:      mov     $0x1,%ebx
      0x0804900a <+10>:     mov     $0x804a000,%ecx
      0x0804900f <+15>:     mov     $0x8,%edx
      0x08049014 <+20>:     int     $0x80
      0x08049016 <+22>:     mov     $0x4,%eax
      0x0804901b <+27>:     mov     $0x1,%ebx
      0x08049020 <+32>:     mov     $0x804a008,%ecx
      0x08049025 <+37>:     mov     $0x7,%edx
      0x0804902a <+42>:     int     $0x80
      0x0804902c <+44>:     mov     $0x1,%eax
      0x08049031 <+49>:     mov     $0x0,%ebx
      0x08049036 <+54>:     int     $0x80
End of assembler dump.
(gdb) █

```

Рисунок 11 Выполнили команду *disassemble*

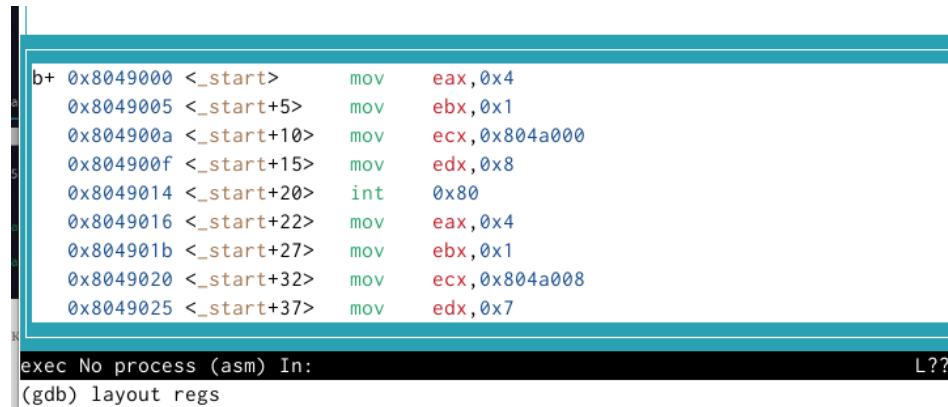
Переключим на отображение команд с Intel'овским синтаксисом, введя команду **set disassembly-flavor intel**.

```
(gdb) set disassembly-flavor intel
```

Рисунок 12 Используем команду *set disassembly-flavor intel*

Перечислим различия отображения синтаксиса машинных команд в режимах АТТ и Intel: они заключаются в порядке операндов и оформлении команд. В синтаксисе Intel используется порядок "назначение-источник", тогда как в АТ&Т применяется обратный порядок "источник-назначение". Кроме того, в АТ&Т-синтаксисе регистры предваряются символом %, а непосредственные значения - символом \$.

Включим режим псевдографики для более удобного анализа программы с помощью **layout asm** (активирует режим разделенного экрана с отображением ассемблерного кода) и **layout regs**.



The screenshot shows a debugger window with two panes. The top pane displays assembly code with addresses and instructions. The bottom pane shows the state of registers.

Address	Instruction
0x8049000 <_start>	mov eax, 0x4
0x8049005 <_start+5>	mov ebx, 0x1
0x804900a <_start+10>	mov ecx, 0x804a000
0x804900f <_start+15>	mov edx, 0x8
0x8049014 <_start+20>	int 0x80
0x8049016 <_start+22>	mov eax, 0x4
0x804901b <_start+27>	mov ebx, 0x1
0x8049020 <_start+32>	mov ecx, 0x804a008
0x8049025 <_start+37>	mov edx, 0x7

exec No process (asm) In: L??
(gdb) layout regs

Рисунок 13 Вводим команды *layout asm* и *layout regs*

2.3. Добавление точек останова

Установить точку останова можно командой **break** (кратко **b**).

С помощью команды **info breakpoints** (кратко **i b**) можно проверить установленные точки останова:

```
(gdb) info breakpoints
Num      Type           Disp Enb Address      What
1        breakpoint    keep y  0x08049000 lab09-2.asm:9
(gdb)
```

Рисунок 14 Используем *i b*

Установим еще одну точку останова по адресу инструкции. Адрес инструкции можно увидеть в средней части экрана в левом столбце соответствующей инструкции. Определим адрес предпоследней инструкции (*mov ebx, 0x0*) и установим точку останова.

```
(gdb) break *0x8049031
Breakpoint 2 at 0x8049031: file lab09-2.asm, line 20.
```

Рисунок 15 Добавили точку останова

Посмотрим информацию о всех установленных точках останова:

```
(gdb) i b
Num      Type           Disp Enb Address      What
1        breakpoint     keep y   0x08049000 lab09-2.asm:9
          breakpoint already hit 1 time
2        breakpoint     keep y   0x08049031 lab09-2.asm:20
```

Рисунок 16 Запрашиваем информацию о точках останова

2.4. Работа с данными программы в GDB

Отладчик может показывать содержимое ячеек памяти и регистров, а при необходимости позволяет вручную изменять значения регистров и переменных.

Посмотрим содержимое регистров с помощью команды **info registers**.

```
ebx      0x0      0
esp      0xffffc330 0xffffc330
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0

B+>0x8049000 <_start>  mov    eax,0x4
0x8049005 <_start+5>   mov    ebx,0x1
0x804900a <_start+10>  mov    ecx,0x804a000
0x804900f <_start+15>  mov    edx,0x8
0x8049014 <_start+20>  int     0x80
0x8049016 <_start+22>  mov    eax,0x4
0x804901b <_start+27>  mov    ebx,0x1
0x8049020 <_start+32>  mov    ecx,0x804a008

native process 4597 (asm) In: _start L9 PC: 0x8049000
--Type <RET> for more, q to quit, c to continue without paging--i r
eip      0x8049000 0x8049000 <_start>
eflags   0x202     [ IF ]
cs       0x23      35
ss       0x2b      43
ds       0x2b      43
es       0x2b      43
fs       0x0       0
gs       0x0       0
(gdb)
```

Рисунок 17 Использовали команду *i r*

Для отображения содержимого памяти можно использовать команду **x**, которая выдаёт содержимое ячейки памяти по указанному адресу. Формат, в котором выводятся данные, можно задать после имени команды через косую черту: **x/NFU**. С помощью команды **x &** также можно посмотреть содержимое переменной. Посмотрим значение переменной *msg1* по имени:

```
(gdb) x/1sb &msg1
0x804a000 <msg1>: "Hello, "
```

Рисунок 18 Посмотрели значение, хранящееся в ячейке *msg1*

Посмотрим значение переменной `msg2` по адресу. Адрес переменной можно определить по дизассемблированной инструкции. Посмотрите инструкцию `mov ecx,msg2` которая записывает в регистр `ecx` адрес переменной `msg2` :

```
(gdb) x/1sb 0x804a008
0x804a008 <msg2>: "world!\n\034"
(gdb)
```

Рисунок 19 Нашли значение `msg2` через адрес

Изменить значение для регистра или ячейки памяти можно с помощью команды `set`, задав ей в качестве аргумента имя регистра или адрес. При этом перед именем регистра ставится префикс `$`, а перед адресом нужно указать в фигурных скобках тип данных . Изменим первый символ переменной `msg1` :

```
(gdb) set {char}0x804a000='h'
(gdb) x/1sb &msg1
0x804a000 <msg1>: "hello, "
(gdb) █
```

Рисунок 20 Изменяем значение регистра

```
(gdb) set {char}0x804a000='h'
(gdb) x/1sb &msg2
0x804a000 <msg2>: "horld!\n\034"
```

Рисунок 21 Изменяем символ в `msg2`

Чтобы посмотреть значения регистров используется команда `print /F <val>` (перед именем регистра обязательно ставится префикс). Выведем в различных форматах (в шестнадцатеричном формате, в двоичном формате и в символьном виде) значение регистра `edx`:

```

(gdb) p/t $ecx
$5 = 100000000010010100000000001000
(gdb) p/s $edx
$6 = 7
(gdb) p/t $edx
$7 = 111
(gdb) p/x $edx
$8 = 0x7
(gdb)

```

Рисунок 22 Вывод регистра в различных форматах

С помощью команды **set** изменим значение регистра *ebx* :

```

(gdb) set $ebx=2
(gdb) p/s $ebx
$4 = 2_

```

Рисунок 23 Изменили значение регистра *ebx*

```

(gdb) set $ebx='2'
(gdb) p/s $ebx
$4 = 50
(gdb)

```

Рисунок 24 Проделали изменение снова

Кавычки ' ' заставляют GDB интерпретировать значение как символ и хранить его ASCII-код, тогда как без кавычек значение трактуется как чистое число.

Завершим выполнение программы с помощью команды **continue** (сокращенно **c**) и выйдем из GDB с помощью команды **quit** .

```

(gdb) continue
Continuing.
hello, world!
Breakpoint 4, _start () at lab09-2.asm:20
(gdb) q
A debugging session is active.

        Inferior 1 [process 10152] will be killed.

Quit anyway? (y or n) █

```

Рисунок 25 Используем *continue* и *quit(q)*

2.5. Обработка аргументов командной строки в GDB

Скопируем файл *lab8-2.asm*, созданный при выполнении лабораторной работы №8, с программой выводящей на экран аргументы командной строки (Листинг 8.2) в файл с именем *lab09-3.asm* и создадим исполняемый файл:

```
vapavlushina@dk7n07 ~/work/arch-pc/lab09 $ cp ~/work/arch-pc/lab08/lab8-2.asm ~/work/arch-pc/lab09/lab09-3.asm
vapavlushina@dk7n07 ~/work/arch-pc/lab09 $ nasm -f elf -g -l lab09-3.lst lab09-3.asm
vapavlushina@dk7n07 ~/work/arch-pc/lab09 $ ld -m elf_i386 -o lab09-3 lab09-3.o
```

Рисунок 26 Копируем и создаём файл

Для загрузки в **gdb** программы с аргументами необходимо использовать ключ **--args**. Загрузим исполняемый файл в отладчик, указав аргументы:

```
vapavlushina@dk7n07 ~/work/arch-pc/lab09 $ gdb --args lab09-3 12 5 3
Exception caught while booting Guile.
Error in function "load-thunk-from-memory":
missing DT_GUILLE_ENTRY
gdb: warning: Could not complete Guile gdb module initialization from:
/usr/share/gdb/guile/gdb/boot.scm.
Limited Guile support is available.
Suggest passing --data-directory=/path/to/gdb/data-directory.
GNU gdb (Gentoo 16.3 vanilla) 16.3
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-pc-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://bugs.gentoo.org/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab09-3...
```

Рисунок 27 Выполняем загрузку с использованием ключа --args

При запуске программы аргументы командной строки загружаются в стек. Исследуем расположение аргументов командной строки в стеке после запуска программы с помощью **gdb**. Для начала установим точку останова перед первой инструкцией в программе и запустим ее.

```
(gdb) b _start
Breakpoint 1 at 0x80490e8: file lab09-3.asm, line 5.
(gdb) run
Starting program: /afs/.dk.sci.pfu.edu.ru/home/v/a/vapavlushina/work/arch-pc/lab09/lab09-3 12 5 3

Breakpoint 1, _start () at lab09-3.asm:5
5      pop ecx ; Извлекаем из стека в `ecx` количество
(gdb) █
```

Рисунок 28 Делаем точку останова

Адрес вершины стека храниться в регистре esp и по этому адресу располагается число равное количеству аргументов командной строки (включая имя программы):

```
(gdb) x/x $esp
0xffffc2d0: 0x00000004
```

Рисунок 29 Адрес вершины стека и кол-во аргументов командной строки.

В данный момент на вершине стека находится число 4.

Посмотрим остальные позиции стека – по адресу [esp+4] располагается адрес в памяти где находится имя программы, по адресу [esp+8] храниться адрес первого аргумента, по адресу [esp+12] – второго и т.д

```
(gdb) x/x $esp
0xffffc2d0: 0x00000004
(gdb) x/x *(void**)(esp + 4)
0xffffc52f: 0x7366612f
(gdb) x/x *(void**)(esp + 8)
0xffffc578: 0x35003231
(gdb) x/x *(void**)(esp + 12)
0xffffc57b: 0x00330035
(gdb) x/x *(void**)(esp + 16)
0xffffc57d: 0x48530033
(gdb) x/x *(void**)(esp + 20)
0x0: Cannot access memory at address 0x0
(gdb)
```

Рисунок 30 Смотрим остальные позиции стека

Объясним, почему шаг изменения адреса равен 4: Шаг изменения адреса равен 4 байта потому, что в данной архитектуре используется 32-битная система с размером машинного слова 4 байта.

3.Задание для самостоятельной работы

1. Преобразуем программу из лабораторной работы №8 (Задание №1 для самостоятельной работы), реализовав вычисление значения функции $f(x)$ как подпрограмму, для этого создадим и оттранслируем файл sama.asm :

```
vapavlushina@dk7n07 ~/work/arch-pc/lab09 $ touch sama.asm
vapavlushina@dk7n07 ~/work/arch-pc/lab09 $ nasm -f elf sama.asm
vapavlushina@dk7n07 ~/work/arch-pc/lab09 $ ld -m elf_i386 -o sama sama.o
vapavlushina@dk7n07 ~/work/arch-pc/lab09 $ ./sama 1 2 3 4
Результат: 256
vapavlushina@dk7n07 ~/work/arch-pc/lab09 $ █
```

Рисунок 31 Запуск программы 1 задания сам.работы


```

#include 'in_out.asm'
SECTION .data
msg db "Результат: ",0
error_msg db "Ошибка: недостаточно аргументов",0
SECTION .text
global _start

; Функция f(x) = 30x - 11
; Вход: eax = x
; Выход: eax = 30*x - 11
f:
    push ebx
    mov ebx, 30      ; ebx = 30
    imul ebx         ; eax = eax * 30 (теперь в eax результат)
    sub eax, 11      ; eax = eax - 11
    pop ebx
    ret

_start:
    pop ecx          ; Извлекаем количество аргументов
    pop edx          ; Извлекаем имя программы

    sub ecx, 1       ; Уменьшаем количество аргументов (без имени прог
    jnz continue    ; Если есть аргументы, продолжаем

    ; Если аргументов нет
    mov eax, error_msg
    call sprintf
    call quit

continue:
    mov esi, 0       ; Используем esi для хранения суммы (esi = 0)

next:
    pop eax          ; Извлекаем следующий аргумент из стека
    call atoi        ; Преобразуем символ в число (результат в eax)

    call f           ; Вычисляем f(x) = 30*x - 11

    add esi, eax      ; Добавляем результат к сумме (esi = esi + f(x))

    loop next        ; Переход к обработке следующего аргумента

_end:
    mov eax, msg      ; Вывод сообщения "Результат: "
    call sprintf
    mov eax, esi      ; Записываем сумму в регистр eax
    call iprintLF     ; Печать результата
    call quit         ; Завершение программы

```

Рисунок 32 Текст программы sata.asm

В листинге 9.3 приведена программа вычисления выражения $(3 + 2) * 4 + 5$. При запуске данная программа дает неверный результат. Создадим файл `sama2.asm`, впишем в него текст листинга 9.3 и проверим это:

```
vapavlushina@dk7n07 ~/work/arch-pc/lab09 $ touch sama2.asm
vapavlushina@dk7n07 ~/work/arch-pc/lab09 $ nasm -f elf sama2.asm
vapavlushina@dk7n07 ~/work/arch-pc/lab09 $ ld -m elf_i386 -o sama2 sama2.o
```

Рисунок 33 Создаём файл

```
vapavlushina@dk7n07 ~/work/arch-pc/lab09 $ ./sama2
Результат: 10
```

Рисунок 34 Запускаем файл

Исправим программу и запустим её:

```
vapavlushina@dk7n07 ~/work/arch-pc/lab09 $ nasm -f elf sama2.asm
vapavlushina@dk7n07 ~/work/arch-pc/lab09 $ ld -m elf_i386 -o sama2 sama2.o
vapavlushina@dk7n07 ~/work/arch-pc/lab09 $ ./sama2
Результат: 25
vapavlushina@dk7n07 ~/work/arch-pc/lab09 $
```

Рисунок 35 Запустили исправленную версию

```
%include 'in_out.asm'
SECTION .data
div: DB 'Результат: ',0
SECTION .text
GLOBAL _start
_start:
; ---- Вычисление выражения (3+2)*4+5
mov ebx,3
mov eax,2
add ebx,eax    ; ebx = 3+2 = 5
mov eax,ebx    ; переносим сумму в eax для умножения
mov ecx,4
mul ecx        ; eax = 5*4 = 20
mov ebx,eax    ; сохраняем результат умножения
add ebx,5      ; ebx = 20+5 = 25
mov edi,ebx
mov eax,div
call sprint
mov eax,edi
call iprintLF
call quit
```

Рисунок 36 Текст исправленной программы

4.Вывод

Приобрела навыки написания программ с использованием подпрограмм.
Ознакомилась с методами отладки при помощи GDB и его основными возможностями.